TP2 : Refactoring SOLID et analyse de packages

Chapitre 2 : Règles de conception logicielle orientée objet

Contexte

Vous êtes ingénieur(e) dans l'équipe **ShopFlow**, une application de commerce en ligne vieille de deux ans. Chaque évolution (nouveau moyen de paiement, nouveau canal de notification, nouvelle règle fiscale) coûte de plus en plus cher. On vous demande un audit de conception, puis un refactoring ciblé.

Principe directeur : appliquer les règles d'or sans sur-ingénierie. Chaque abstraction introduite doit être justifiée par un besoin réel (YAGNI).

Partie 1 : SOLID

Code existant

```
public class OrderProcessor {
    public double process(Order order, String paymentType, String notifType) {
        double total = 0;
        for (Item i : order.getItems()) total += i.getPrice() * i.getQty();
        if (order.getCustomerType().equals("VIP")) total *= 0.9;

        switch (paymentType) {
            case "CREDIT_CARD": System.out.println("Débit CB : " + total); break;
            case "PAYPAL":      System.out.println("PayPal : " + total);   break;
            case "CASH":        System.out.println("Espèces : " + total);  break;
            default: throw new IllegalArgumentException(paymentType);
        }

        MySqlDatabase db = new MySqlDatabase("jdbc:mysql://localhost/shop");
        db.save(order, total);

        if (notifType.equals("EMAIL"))
            new SmtpClient().send(order.getEmail(), "Commande validée");
        else if (notifType.equals("SMS"))
            new SmsGateway().send(order.getPhone(), "Commande validée");
        return total;
    }
}
```

```
public interface PaymentMethod {
    void pay(double amount);
    void refund(double amount);
    void recurringCharge(double amount);   // abonnements
}

public class CashPayment implements PaymentMethod {
    public void pay(double a)             { /* encaissement */ }
    public void refund(double a)          { /* remboursement */ }
    public void recurringCharge(double a) { throw new UnsupportedOperationException(); }
}
```

1.1 Diagnostic

Complétez le tableau pour `OrderProcessor` et `PaymentMethod` :

Symptôme (rigidité, fragilité, immobilité, viscosité, répétition, opacité, complexité)	Ligne ou zone concernée	Principe SOLID violé

Symptôme (rigidité, fragilité, immobilité, viscosité, répétition, opacité, complexité)	Ligne ou zone concernée	Principe SOLID violé

1.2 Refactoring SRP, OCP, DIP

Refactorez `OrderProcessor` pour que :

1. **SRP:** Chaque responsabilité soit isolée dans sa propre classe (calcul du total et remise, paiement, persistance, notification).
2. **OCP**: L'ajout d'un moyen de paiement `CRYPTO` ne nécessite **aucune modification** de `OrderProcessor`.
3. **DIP :** `OrderProcessor` ne dépende que d'abstractions (`OrderRepository`, `Notifier`, `PaymentMethod`, `DiscountPolicy`), injectées par constructeur.

Livrable : Code Refactorisé.

1.3 LSP et ISP

1. **LSP :** `CashPayment` respecte-t-elle le contrat de `PaymentMethod` ? Quel code client existant peut échouer à l'exécution ?
2. **ISP**: Proposez une découpe de `PaymentMethod` en interfaces cohérentes (par client, pas par commodité).
3. Quel est le risque inverse si vous créez une interface par méthode ? Justifiez votre découpage en une phrase.

1.4 Validation

Ajoutez `CryptoPayment` et un canal de notification `PushNotifier` sans toucher à `OrderProcessor`. Notez le nombre de fichiers modifiés et de fichiers ajoutés.

Partie 2 : cohésion et couplage de packages

Architecture actuelle de ShopFlow

Une flèche `A → B` signifie « A dépend de B ». Na = classes abstraites et interfaces, Nc = nombre total de classes.

Package	Nc	Na	Dépend de
`ui`	12	0	`orders`, `notifications`
`orders`	10	2	`billing`, `persistence`
`billing`	8	1	`orders`, `persistence`
`notifications`	6	0	`orders`
`persistence` (JDBC)	9	0	(aucun)

2.1 Métriques

Calculez pour chaque package : Ca (couplage afférent), Ce (couplage efférent), l'instabilité `I = Ce / (Ca + Ce)`, l'abstraction `A = Na / Nc` et la distance à la séquence principale `D = |A + I - 1|`.

Package	Ca	Ce	I	A	D
`ui`					
`orders`					
`billing`					
`notifications`					
`persistence`					

2.2 Diagnostic ADP, SDP, SAP (Couplage)

1. **ADP :** repérez le cycle. Quelle conséquence en équipe (« syndrome du lendemain matin ») ? Proposez deux corrections et choisissez-en une.
2. **SDP :** listez les dépendances qui pointent vers un package moins stable que l'appelant.
3. **SAP :** quel package se trouve en zone de douleur (stable et concret) ? Pourquoi est-ce un problème pour ShopFlow ?
4. Dessinez le graphe corrigé : toutes les flèches doivent aller vers plus de stabilité, sans cycle.

2.3 Diagnostic REP, CCP, CRP (Cohésion)

Pour chaque scénario, identifiez le principe en jeu, le coût actuel et la réorganisation proposée.

Scénario	Principe	Coût actuel	Réorganisation
A. Une nouvelle loi fiscale impose de modifier `TaxCalculator` (`billing`), `InvoicePrinter` (`ui`) et `AccountingExport` (`exports`).			
B. L'application « Calculatrice Mobile » importe le package `core` uniquement pour `MoneyUtils`, alors que `core` embarque aussi les drivers SQL.			
C. `orders` est utilisé par l'équipe mobile sans numéro de version. Vous changez le calcul de la remise.			

Question de synthèse : pour une startup au stade MVP, quel principe faut-il favoriser, et vers quoi refactoriser à maturité ? Quelle tension les trois principes créent-ils entre eux ?